

Лабораторна робота № 9

з курсу "Архітектура обчислювальних систем та комп'ютерна схемотехніка"

Виконав:

Студент групи ПМі-12

Козій Дем'ян

Тема: Команди передачі управління та організація циклів в Assembler.

Мета роботи: Використовуючи середовище Visual Studio, на прикладі обчислення математичного виразу освоїти програмування з використанням команд переходу та циклів на мові Assembler.

Варіант: 11

Хід виконання роботи:

1. У середовищі Visual Studio було створено C++ проект та підключено інструмент Microsoft Macro Assembler (MASM) через налаштування Build Customizations.
2. Було реалізовано програму для обчислення кусково-заданої функції $y = y1 \bmod \{y2\}$ згідно з варіантом №11 для масиву з п'яти цілих значень x .
3. Для реалізації циклічного обчислення використано лічильник `ecx` (встановлено значення 5) та команду `loop`, яка автоматично декрементує регістр і виконує перехід до мітки початку ітерації, поки `ecx > 0`.
4. Для переміщення по масиву `x_values` та `y_values` застосовано базово-індексну адресацію через регістр `esi`, який збільшувався на 4 байти (розмір типу `int`) кожної ітерації.
5. Розгалуження в обчисленнях `y1` та `y2` виконано за допомогою команд порівняння (`cmp`) та умовних переходів (`jg` — перехід, якщо більше).
6. Для знаходження абсолютної величини модуля $|x|$ без розгалужень застосовано алгоритм розширення знаку (`cdq`), побітового виключного АБО (`xor eax, edx`) та віднімання (`sub eax, edx`).
7. Остаточне обчислення остачі від ділення виконано за допомогою команди ділення зі знаком `idiv`. Результат (остача), збережений у регістрі `edx`, успішно записано у вихідний масив.

Програмний код:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    // Параметри для прикладу з методички
    int a = -6, b = 4, c = 8, d = 2; //
    // Параметр для індивідуального завдання
    int a_var11 = 5;

    // Масив значень x (від -2 до 2)
    int x_values[5] = { -2, -1, 1, 2, 4 };
    int y_example[5]; // Результати прикладу
    int y_variant[5]; // Результати вар. 11

    __asm {
        mov esi, 0          // Індекс у байтах
        mov ecx, 5          // Лічильник циклу

        start_loop:
        mov ebx, x_values[esi] // Поточне x

        // === 1. ОБЧИСЛЕННЯ ПРИКЛАДУ (З МЕТОДИЧКИ) ===
        // Умови: x < 1; x = 1; x > 1
        cmp ebx, 1
        jl label_l1         // x < 1
        je label_equals      // x = 1

        // Випадок x > 1: y = (a^2 - x) / c
        mov eax, a
        imul eax            // eax = a^2
        sub eax, ebx        // eax = a^2 - x
        cdq
        mov edi, c
        idiv edi            // eax = (a^2 - x) / c
        jmp save_example

        label_l1 : // Випадок x < 1: y = 2ax + 5
        mov eax, 2
        imul a              // eax = 2a
        imul ebx            // eax = 2ax
        add eax, 5          // eax = 2ax + 5
        jmp save_example

        label_equals : // Випадок x = 1: y = (a - b) / d
        mov eax, a
        sub eax, b          // eax = a - b
        cdq
        mov edi, d
        idiv edi            // eax = (a - b) / d

        save_example :
        mov y_example[esi], eax

        // === 2. ОБЧИСЛЕННЯ ВАРІАНТУ 11 ===
        // y = y1 mod y2

        // --- Обчислення y2 (збережемо в edi) ---
        cmp ebx, 4
        jg y2_gt_4
        mov edi, ebx        // x <= 4 -> y2 = x
        jmp compute_y1
        y2_gt_4 :
        mov edi, 2          // x > 4 -> y2 = 2

        compute_y1 :
        // --- Обчислення y1 (збережемо в eax) ---
        cmp ebx, 1
        jg y1_gt_1
        // x <= 1 -> y1 = |x| + a
        mov eax, ebx
        cdq
        xor eax, edx
        sub eax, edx        // |x|
        add eax, a_var11    // |x| + a
        jmp do_mod
        y1_gt_1 :
        // x > 1 -> y1 = 10 + x
        mov eax, 10
        add eax, ebx

        do_mod :
        // --- y1 mod y2 ---
        cdq
        idiv edi            // остача в edx
        mov y_variant[esi], edx

        // ПЕРЕХІД ДО НАСТУПНОЇ ІТЕРАЦІЇ
        add esi, 4
        loop start_loop
    }

    // Виведення результатів
    cout << "-----" << endl;
    cout << " x | Завдання 1 | Завдання 2 (y1 mod y2)" << endl;
    cout << "-----" << endl;
    for (int i = 0; i < 5; i++) {
        cout << setw(2) << x_values[i] << " | "
            << setw(11) << y_example[i] << " | "
            << setw(16) << y_variant[i] << endl;
    }

    return 0;
}
```

Результати виконання програми:

х	Завдання 1	Завдання 2 ($y1 \bmod y2$)
-2	29	1
-1	17	0
1	-5	0
2	4	0
4	4	2

Висновок:

Під час виконання лабораторної роботи № 9 було успішно засвоєно принципи організації циклічних процесів та складних розгалужень на мові Assembler. На практиці досліджено роботу команди `loop`, використання регістра `ecx` як лічильника та застосування регістрів загального призначення (зокрема `esi`) для покрокового доступу до елементів масивів пам'яті. Розроблена програма коректно обчислює задану багатокрокову математичну функцію для цілого діапазону вхідних даних та обробляє модульну арифметику з урахуванням знаків.